

21720 Programació concurrent

3. Algorismes avançats per a sistemes de memòria
compartida

Solucions avançades al problema de la regió crítica

- Solucions per a un nombre arbitrari de processos
- Impliquen característiques addicionals relatives a l'abstracció de la concurrència i a l'especificació del problema de la regió crítica
- Solucions que no impliquen cap suport específic de *hardware* o de sistema operatiu
- Més que utilitat pràctica tenen interès acadèmic en la comprensió del problema i les solucions

Algorisme del forn (*bakery*) (Lamport 1974)

- El nom és degut al símil del comerç on el client (procés) agafa número per ser atès (entrar a la SC)
- El procés espera fins que el seu número és el menor de tots els que esperen i llavors entra a la SC

Algorithm 5.1: Bakery algorithm (two processes)	
integer $np \leftarrow 0$, $nq \leftarrow 0$	
p	q
loop forever p1: non-critical section p2: $np \leftarrow nq + 1$ p3: await $nq = 0$ or $np \leq nq$ p4: critical section p5: $np \leftarrow 0$	loop forever q1: non-critical section q2: $nq \leftarrow np + 1$ q3: await $np = 0$ or $nq < np$ q4: critical section q5: $nq \leftarrow 0$

- Solució simplificada per a 2 processos
- np i nq són el número dels dos processos
- Un valor 0 implica que el procés no vol entrar a la SC i un positiu la cua de processos que volen entrar. Com més petit més proper al cap de la cua
- Si són iguals s'assigna precedència a p
- L'algorisme satisfà l'exclusió mútua i no presenta interbloqueig. Es suposa $p2$ i $q2$ són atòmics (no és realista)

Algorithm 5.2: Bakery algorithm (N processes)

integer array[1..n] number $\leftarrow$ [0,...,0]

loop forever

p1: non-critical section

p2: number[i] $\leftarrow$ 1 + max(number)

p3: for all *other* processes j

p4: await (number[j] = 0) or (number[i] $\ll$ number[j])

p5: critical section

p6: number[i] $\leftarrow$ 0

- La mida de l'*array number* ha de ser el nombre màxim de processos concurrents. Cada procés està identificat amb la posició dins l'*array*
- La funció *max* retorna el nombre més gran de *number*. A cada procés se li assigna el seu torn
- Es fa un recorregut de *number* per a tots els processos j
- Espera si el nombre de j és menor al d' i

- La notació $(\text{number}[i] \ll \text{number}[j])$ és una abreviació de:
 $(\text{number}[i] < \text{number}[j]) \text{ or } ((\text{number}[i] = \text{number}[j]) \text{ and } (i < j))$
- La funció *max* no és atòmica: Pot donar el mateix torn (empat)
- Si els torns són iguals es prioritza el procés amb identificador menor (desempat)
- Si sempre hi ha un procés a la SC el nombre de torn serà il·limitat
- Cada procés fa el recorregut de l'*array* inclús si cap procés vol entrar a la SC

- L'algorisme no satisfà l'**exclusió mútua**:

Pot passar que $p1$ arribi al bucle *for j* mentre $p0$ encara no hagi enregistrat el seu $number[0]$, de manera que $p1$ trobarà $number$ amb $[0, 1, 0, \dots, 0]$ llavors la condició $(number[0]=0)$ serà *true* i $p1$ entrarà a la SC

Poc després $p0$ emmagatzemarà $number[0]=1$ i $[1, 1, 0, \dots, 0]$ per tant la condició $((number[i]=number[j]) \text{ and } (i < j))$ és *true* i $p0$ també entra a la SC

Algorithm 5.3: Bakery algorithm without atomic assignment

boolean array[1..n] choosing $\leftarrow$ [false,...,false]

integer array[1..n] number $\leftarrow$ [0,...,0]

loop forever

p1: non-critical section

p2: choosing[i] $\leftarrow$ true

p3: number[i] $\leftarrow$ 1 + max(number)

p4: choosing[i] $\leftarrow$ false

p5: for all *other* processes j

p6: await choosing[j] = false

p7: await (number[j] = 0) or (number[i] $\ll$ number[j])

p8: critical section

p9: number[i] $\leftarrow$ 0

- Si *choosing[i]* és true implica que el procés *i* està escollint nombre i els altres processos han d'esperar a que l'elecció estigui feta abans de comparar els nombres

- Aquest algorisme satisfà l'**exclusió mútua**
- Dos processos haurien de tenir el mateix nombre per poder estar a l'hora a la SC, l'ús d'identificador únic garanteix que un serà menor i per tant el que sortirà del bucle que compara i i j
- Si $p1$ és a la SC i $p2$ vol entrar ha tenir id. menor que $p1$ i això no pot passar: Quan $p1$ surt de *choosing* si $p2$ encara no ha entrat quan entri tindrà un nombre major i si ja n'ha sortit vol dir $p2$ ha entrat abans a la SC

- Aquest algorisme no presenta **interbloqueig**:
 - El pitjor cas és que dos processos tinguin el mateix nombre, en aquest cas es prioritza el que té ID menor
- No presenta **espera infinita** :
 - Un procés que surti de la SC i vulgui tornar a entrar tindrà el nombre major que tots els que esperen. Si n processos volen entrar a l'hora hauran d'esperar com a màxim $n-1$ torns

backery.c

```
for(i=0; i<NUM_THREADS; i++){
    id[i].tid = i;
    rc = pthread_create(&threads[i], NULL, count, (void *) &id[i]);
}
```

```
void *count(void *ptr) {
    long i, max = MAX_COUNT/NUM_THREADS;
    int tid = ((struct tdata *) ptr)->tid;
    for (i=0; i < max; i++) {
        lock(tid);
        counter += 1;
        unlock(tid);
    }
    printf("End %d counter: %d\n", tid, counter);
}
```

```
void lock(int i) {
    int j;
    choosing[i] = 1;
    __atomic_thread_fence(__ATOMIC_RELEASE);
    number[i] = max(number, NUM_THREADS) + 1;
    __atomic_thread_fence(__ATOMIC_SEQ_CST);
    choosing[i] = 0;
    for (j=0; j<NUM_THREADS; j++) {
        while(choosing[j]);
        while(number[j] > 0 &&
            (number[j] < number[i] ||
             (number[j] == number[i] && j < i))){
            sched_yield(); // Si no bloqueja!!!
        }
    }
}
```

```
void unlock(int i) {
    __atomic_thread_fence(__ATOMIC_SEQ_CST);
    number[i] = 0;
}
```

CounterBakery.java

```
Thread[] threads = new Thread[THREADS];
choosing = new boolean[THREADS];
number = new int[THREADS];
int i;
for (i = 0; i < THREADS; i++) {
    choosing[i] = false;
    number[i] = 0;
    threads[i] = new Thread(new CounterBakery(i));
    threads[i].start();
}
for (i = 0; i < THREADS; i++) {
    threads[i].join();
}
```

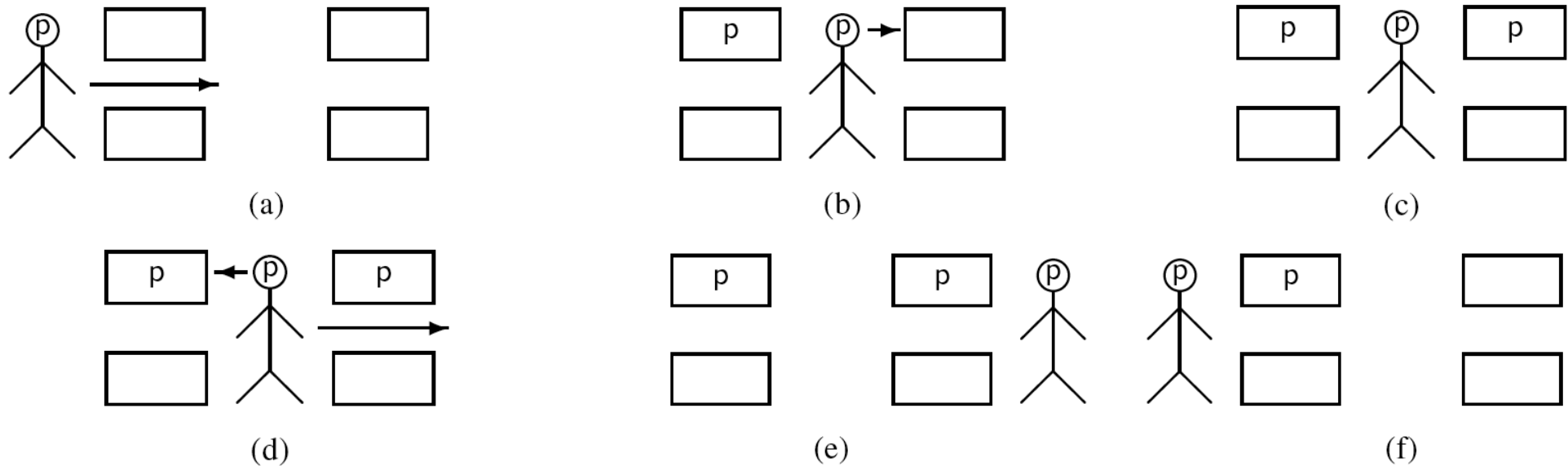
```
@Override
public void run() {
    int fi = MAX_COUNT / THREADS;
    System.out.println("Thread " + id);
    for (int i = 0; i < fi; i++) {
        // Bakery
        choosing[id] = true;
        number[id] = max(number) + 1;
        choosing[id] = false;
        for (int j = 0; j < number.length; j++) {
            if (j != id) { // for all other processes j
                while (choosing[j]) {
                    Thread.yield(); //Pausa el fill
                }
                while (number[j] != 0 && (number[id] > number[j]
                    || (number[id] == number[j] && id > j))) {
                    Thread.yield(); // Si no el processador no permet sortir del bucle
                }
            }
        }
        n += 1; //SC
        number[id] = 0;
    }
}
```

Algorisme ràpid de Lamport (1987)

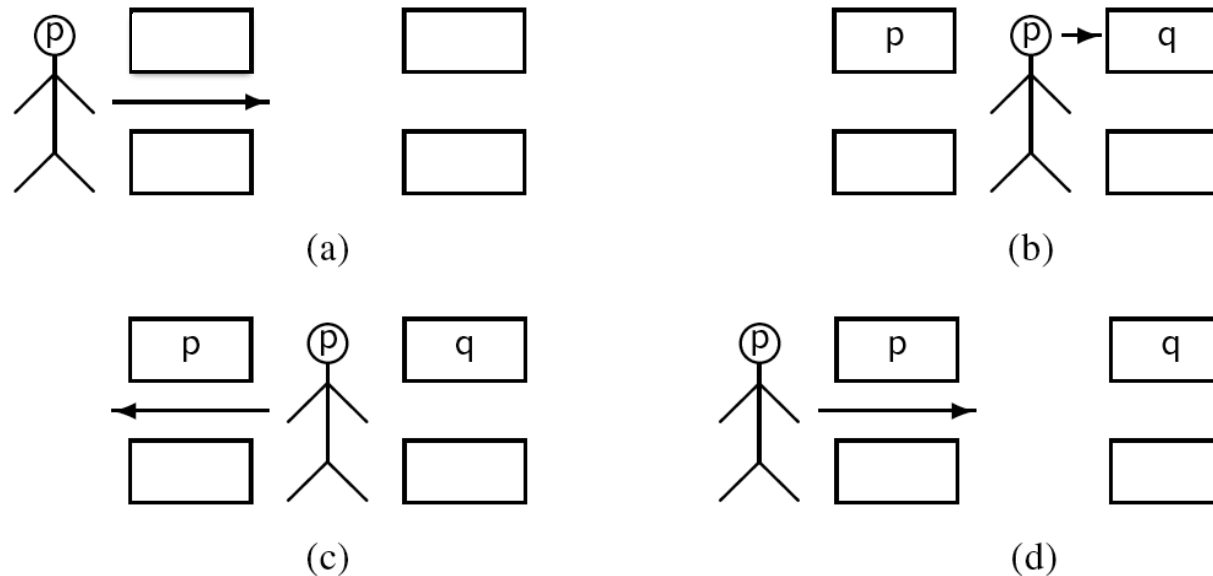
- Problemes de l'algorisme del forn:
 - Requereix $2n$ registres de memòria per als 2 *arrays choosing i number*
 - Malgrat no hi hagi competència l'algorisme fa el recorregut d'ambdós

- Esquema de la simplificació de l'algorisme per a dos processos
- Es suposa que p i q són identificadors numèrics

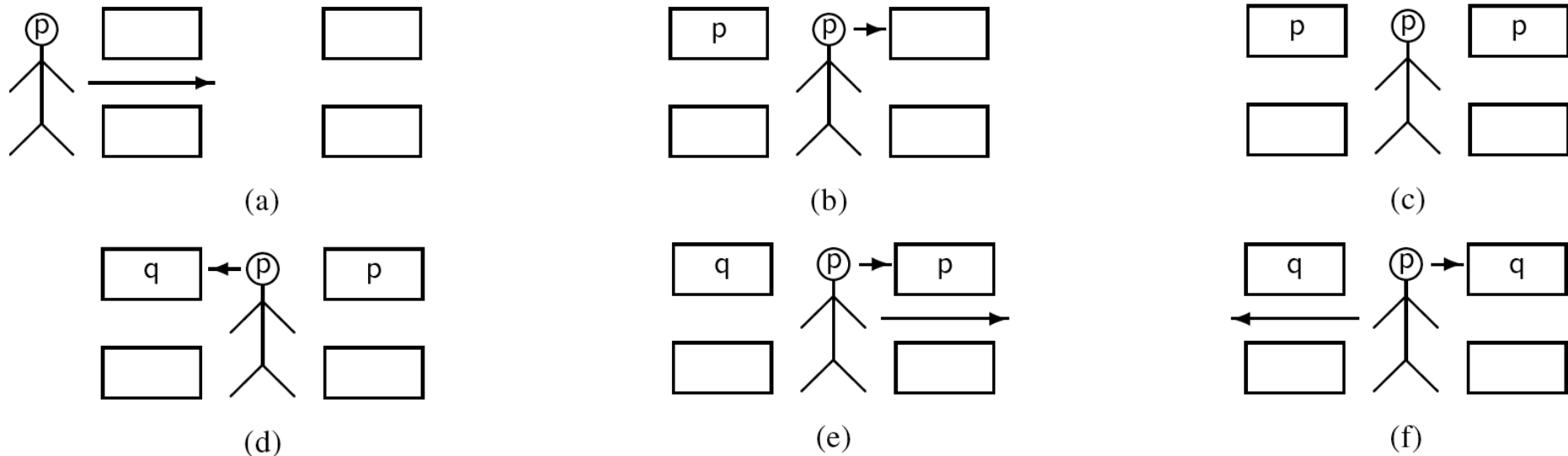
Algorithm 5.4: Fast algorithm for two processes (outline)	
integer gate1 $\leftarrow$ 0, gate2 $\leftarrow$ 0	
p	q
loop forever non-critical section p1: gate1 $\leftarrow$ p p2: if gate2 $\neq$ 0 goto p1 p3: gate2 $\leftarrow$ p p4: if gate1 $\neq$ p p5: if gate2 $\neq$ p goto p1 critical section p6: gate2 $\leftarrow$ 0	loop forever non-critical section q1: gate1 $\leftarrow$ q q2: if gate2 $\neq$ 0 goto q1 q3: gate2 $\leftarrow$ q q4: if gate1 $\neq$ q q5: if gate2 $\neq$ q goto q1 critical section q6: gate2 $\leftarrow$ 0



- Esquema de quan **no hi ha conflicte**. La NoSC és a l'esquerra de cada diagrama i la SC a la dreta, els quadres simbolitzen portes i l'home el procés
- El procés entra a la primera porta i hi escriu l'ID [$p1$, (a)]
- Mira la segona porta [$p2$, (b)]. Si no hi ha competència no hi ha res escrit en ella
- El procés escriu el seu ID a la segona porta [$p3$, (c)]
- Mira darrera a la porta 1 si el seu ID encara hi és [$p4$, (d)]
- Si és així entra a la SC [$p6$, (e)], abans d'anar-se'n esborra l'ID de la segona porta [$p6$, (f)]



- Esquema amb **conflicte a la porta 2**
- El procés entra a la primera porta [$p1$, (a)] però quan mira a la segona veu que una altre procés ja hi ha escrit l'ID [$p2$, (b)]
- La SC està ocupada per q per tant p surt abans de la primera porta [$p2$, (c)]
- I ho intenta de bell nou [$p1$, (d)]



- Esquema amb **conflicte a la porta 1**
- Inicialment el procés fa el mateix [$p1$, (a)], [$p2$, (b)], [$p3$, (c)]
- Però quan es gira veu que el procés q ha entrat a la porta 1 [$p4$, (d)]
- Hi ha dues possibilitats: Si l'identificador de p es manté a la segona porta, continua a la SC [$p5$, (e)]. O si el procés q ha passat a p i ha escrit el seu ID a la segona porta, en aquesta cas p torna a la primera [$p5$, (f)]

Algorithm 5.6: Fast algorithm for two processes

integer gate1 $\leftarrow$ 0, gate2 $\leftarrow$ 0

boolean wantp $\leftarrow$ false, wantq $\leftarrow$ false

p

p1: gate1 $\leftarrow$ p
 wantp $\leftarrow$ true

p2: if gate2 $\neq$ 0
 wantp $\leftarrow$ false
 goto p1

p3: gate2 $\leftarrow$ p
p4: if gate1 $\neq$ p
 wantp $\leftarrow$ false
 await wantq = false

p5: if gate2 $\neq$ p goto p1
 else wantp $\leftarrow$ true
 critical section

p6: gate2 $\leftarrow$ 0
 wantp $\leftarrow$ false

q

q1: gate1 $\leftarrow$ q
 wantq $\leftarrow$ true

q2: if gate2 $\neq$ 0
 wantq $\leftarrow$ false
 goto q1

q3: gate2 $\leftarrow$ q
q4: if gate1 $\neq$ q
 wantq $\leftarrow$ false
 await wantp = false

q5: if gate2 $\neq$ q goto q1
 else wantq $\leftarrow$ true
 critical section

q6: gate2 $\leftarrow$ 0
 wantq $\leftarrow$ false

- Per entrar a la SC (p) s'ha de verificar que

$$gate2 = p \rightarrow \neg((gate2 = q) \vee (gate1 \neq q) \vee (gate2 = 0))$$

Per això és necessari el *wantp* (i el *wantq*)

- Generalització per N processos
- Substitució de `await wantq=false` per una repetició per a tots els altres processos

```
for all other processes j
    await want[j] = false
```

fast.c

```
unsigned int want[NUM_THREADS];
int gate1 = -1, gate2 = -1;

void lock(int i) {
    int j;

start:
    gate1 = i;
    want[i] = 1;
    __atomic_thread_fence(__ATOMIC_SEQ_CST);
    if (gate2 >= 0) {
        want[i] = 0;
        while (gate2 >= 0);
        goto start;
    }
    gate2 = i;
    __atomic_thread_fence(__ATOMIC_SEQ_CST);
    if (gate1 != i) {
        want[i] = 0;
        for (j = 0; j < NUM_THREADS; j++) {
            while (want[j]);
        }
        if (gate2 != i) {
            goto start;
        } else want[i] = 1;
    }
}

void unlock(int i) {
    __atomic_thread_fence(__ATOMIC_SEQ_CST);
    gate2 = -1;
    want[i] = 0;
}
```

Consistència seqüencial

```
P003_Dekker$cc -pthread dekkerNoSinc.c
P003_Dekker$time ./a.out
Final result: 99349195 Expected: 100000000 Error: 0.651%
Bye from main
```

```
real    0m2.115s
user    0m4.155s
sys     0m0.005s
```

- Els algorismes anteriors són correctes però no funcionen bé en la majoria de processadors moderns: Increment de temps de CPU i no se compleix l'exclusió mútua
- Els processadors moderns no garanteixen que les instruccions s'executin en el mateix ordre del programa: No hi ha consistència seqüencial d'accés a memòria

- **Motius** de la no consistència seqüencial:
 - Optimitzacions del compilador. Per evitar-les es poden usar variable *volatile*
 - Incoherència de caché de RAM a multiprocessadors: Responsable de l'increment de temps per la sobrecàrrega a l'accés a variables compartides
 - Execució fora d'ordre o execució dinàmica: Els processadors reordenen les instruccions per estalviar cicles de CPU

Barreres de memòria

https://gcc.gnu.org/onlinedocs/gcc/_005f_005fatomic-Builtins.html

- Solució a la inconsistència seqüencial
- Indicacions als processador:
 - Abans de continuar s'han d'executar totes les operacions L/E anteriors a la barrera
 - Cap operació de L/E posteriors es pot executar abans de la barrera
- Les barreres tenen un cost de CPU elevat: Interessa trobar el mínim

<http://www.rdrop.com/users/paulmck/scalability/paper/whymb.2010.07.23a.pdf>